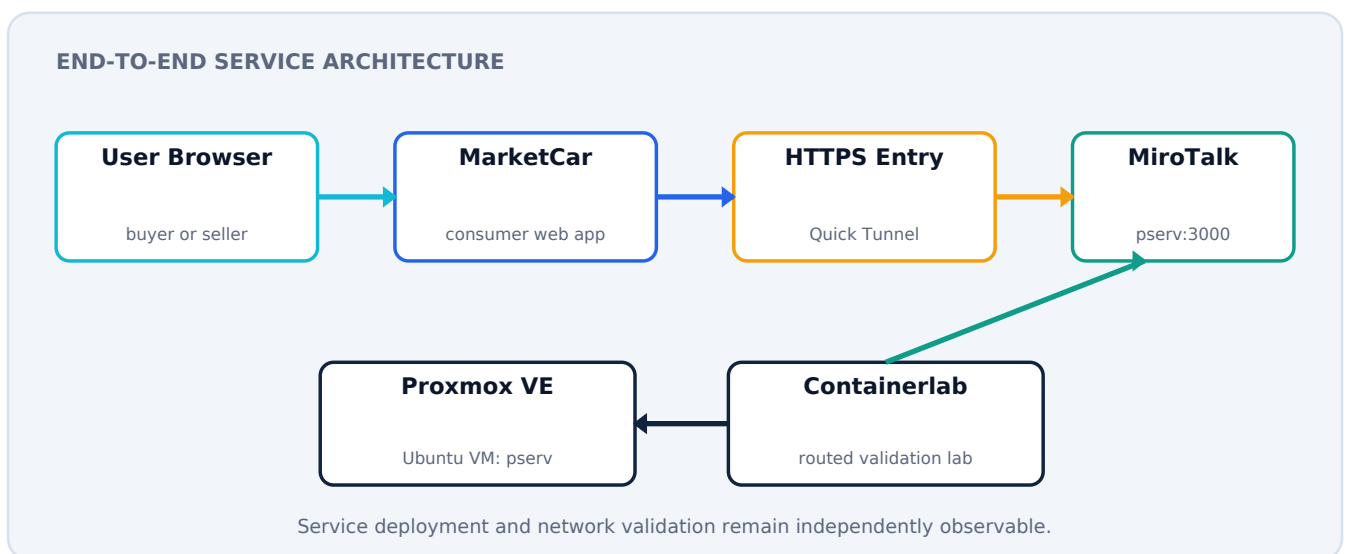


# Self-Hosted MiroTalk P2P over a Routed Containerlab Topology

## with MarketCar Service Integration

Design, deployment, routing validation, secure browser access, and application-level integration of a real-time WebRTC service.



|             |                              |
|-------------|------------------------------|
| Prepared by | Izz Aldeen Mansour           |
| Institution | An-Najah National University |
| Course      | Network Programming          |
| Date        | August 2026                  |

# Abstract and Project Scope

This project implements a complete network service rather than a standalone web page. MiroTalk P2P is self-hosted on an Ubuntu Server virtual machine named **pserv**, running under Proxmox VE. A routed Containerlab environment is used to model two client networks and verify that both clients can reach the service through virtual routers. A separate web application, MarketCar, consumes the meeting service by generating listing-specific room URLs and opening MiroTalk directly from a vehicle listing.

The implementation combines virtualization, containers, routing, service exposure, WebRTC browser requirements, and application integration. Local service checks and client-side HTTP tests returned **HTTP/1.1 200 OK**. External demonstration access is provided through a Cloudflare Quick Tunnel, which supplies a temporary HTTPS endpoint required for reliable browser permission requests for camera and microphone access.

## Project objectives

- Deploy a real self-hosted WebRTC application on a server that is independent of the development laptop.
- Create an explicit routed topology with two client subnets and verify end-to-end service reachability.
- Expose the service through HTTPS for a practical two-device video meeting demonstration.
- Integrate MiroTalk with a second application so that the server acts as a reusable communication service provider.

## Implemented scope

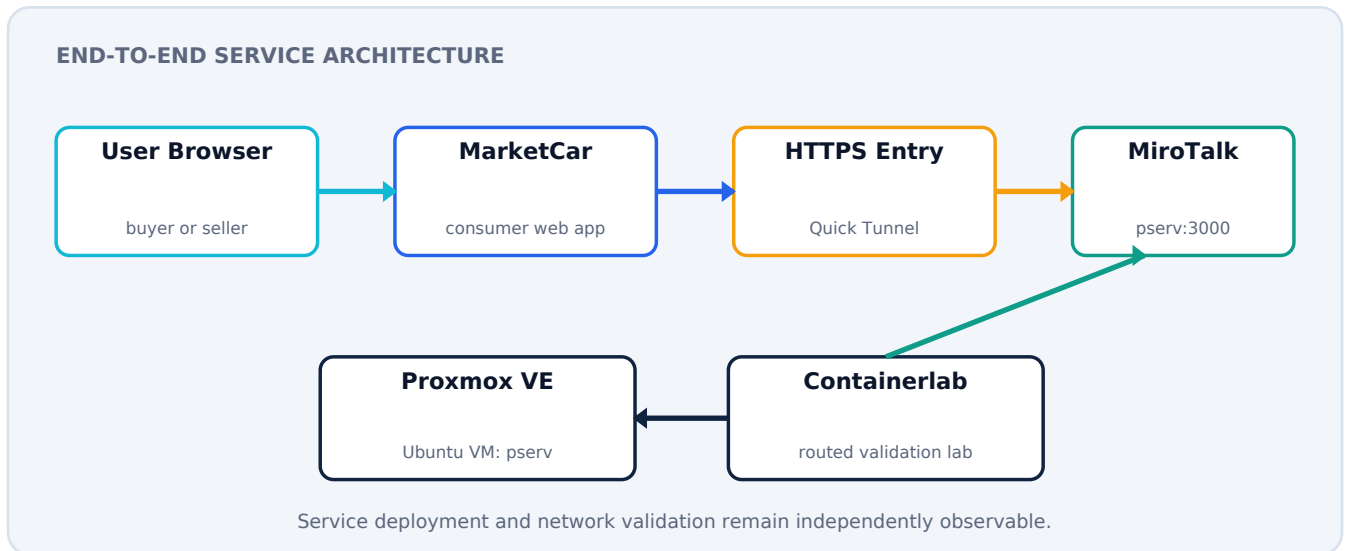
| Layer          | Implemented component                      | Purpose                                               |
|----------------|--------------------------------------------|-------------------------------------------------------|
| Infrastructure | Proxmox VE + Ubuntu Server VM              | Persistent host for the application and network lab   |
| Application    | MiroTalk P2P in Docker                     | Video, audio, chat, room creation, and screen sharing |
| Network        | Containerlab with client1, r1, r2, client2 | Routed reachability across isolated IP networks       |
| Integration    | MarketCar static web application           | Launch a room from a selected vehicle listing         |
| Secure access  | Cloudflare Quick Tunnel                    | Temporary public HTTPS endpoint for demonstration     |

### Main contribution

The result is an end-to-end demonstrator in which a consumer application invokes a real-time communication service hosted on separate server infrastructure and validated through a controlled routed lab.

## Architecture and Deployment Environment

The system is divided into three observable planes. The **service plane** contains MiroTalk on the Ubuntu VM. The **network plane** contains the Containerlab clients, routers, interfaces, and routes. The **integration plane** contains MarketCar and the generated meeting URL. This separation makes failures easier to isolate: the service can be checked locally before routing is tested, and routing can be tested before the browser integration is demonstrated.



| Item                    | Final configuration              |
|-------------------------|----------------------------------|
| Physical host role      | Standalone Proxmox VE server     |
| Guest system            | Ubuntu Server VM, hostname pserv |
| VM address              | 172.16.107.5                     |
| MiroTalk directory      | ~/network-project/mirotalk       |
| Application image       | mirotalk/p2p:latest              |
| Container name and port | mirotalk, TCP 3000               |
| Network lab directory   | ~/network-project/mirotalk-lab   |
| Topology definition     | mirotalk-final.clab.yml          |

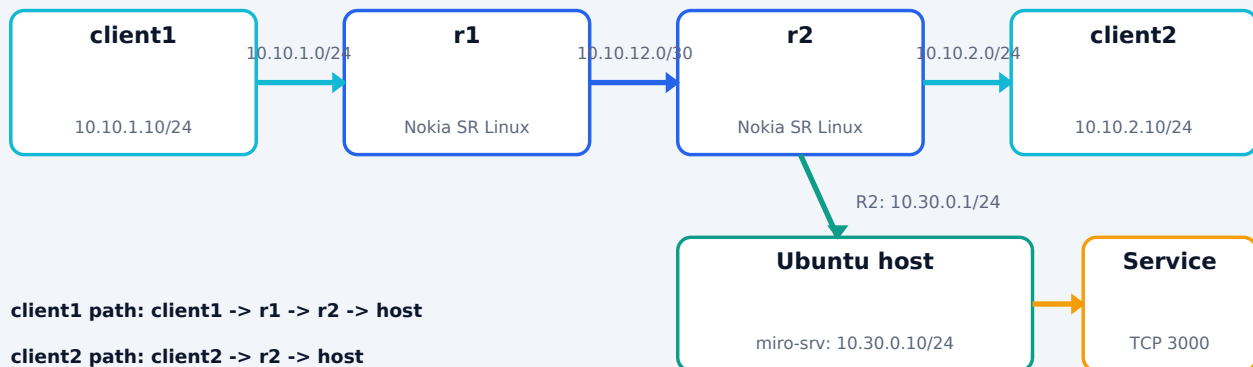
### Why this architecture

- Proxmox keeps the service available on dedicated infrastructure and separates it from the Windows presentation laptop.
- Docker packages MiroTalk and its runtime into a repeatable service deployment.
- Containerlab creates explicit nodes and virtual links from a topology file and manages the lab lifecycle [2].
- MiroTalk was selected because it is a self-hosted WebRTC P2P platform with browser-based rooms and Docker deployment support [1].

## Containerlab Topology and IP Plan

The final network consists of two browser clients and two routed nodes. Client1 reaches the server through both routers; client2 reaches it through r2. The Ubuntu host is attached to the server-side network through interface **miro-srv**. This design creates a visible multi-hop path rather than placing every node in one broadcast domain.

### FINAL ROUTED TOPOLOGY



| Segment     | Endpoint    | Address       | Role                  |
|-------------|-------------|---------------|-----------------------|
| Client1 LAN | client1     | 10.10.1.10/24 | Browser client        |
| Client1 LAN | r1          | 10.10.1.1/24  | Default gateway       |
| Transit     | r1          | 10.10.12.1/30 | Router-to-router link |
| Transit     | r2          | 10.10.12.2/30 | Router-to-router link |
| Client2 LAN | client2     | 10.10.2.10/24 | Browser client        |
| Client2 LAN | r2          | 10.10.2.1/24  | Default gateway       |
| Server LAN  | r2          | 10.30.0.1/24  | Gateway toward host   |
| Server LAN  | Ubuntu host | 10.30.0.10/24 | MiroTalk endpoint     |

### Host routing

The Ubuntu host uses routes for 10.10.1.0/24 and 10.10.2.0/24 via 10.30.0.1, allowing return traffic to reach both client networks.

## 4.1 MiroTalk service deployment

MiroTalk runs as a Docker Compose service on pserv. The source-based deployment was prepared from the project templates, including **.env**, **app/src/config.js**, and **docker-compose.yml**. The official project also documents Docker-based deployment and configuration through these files [1].

```
cd ~/network-project/mirotalk
docker compose up -d
docker ps
curl -I http://127.0.0.1:3000
```

## 4.2 Network lab deployment

The topology is defined in **mirotalk-final.clab.yml**. Router configuration is separated into CLI files, while host interface and route preparation are handled by scripts. This keeps deployment repeatable and makes the topology easier to inspect during the demonstration.

| Artifact                | Responsibility                                                    |
|-------------------------|-------------------------------------------------------------------|
| mirotalk-final.clab.yml | Defines nodes, images, links, port mappings, and startup behavior |
| configs/r1.cli          | Configures r1 interfaces and routing                              |
| configs/r2.cli          | Configures r2 interfaces and routing                              |
| scripts/host-up.sh      | Creates the host-side interface, routes, and lab prerequisites    |
| scripts/host-down.sh    | Removes host-side lab state                                       |
| scripts/test-lab.sh     | Runs the repeatable reachability checks                           |

```
cd ~/network-project/mirotalk-lab
./scripts/host-up.sh
sudo containerlab inspect -t mirotalk-final.clab.yml
docker ps --format 'table {{.Names}}\t{{.Image}}\t{{.Status}}'
```

## 4.3 Browser clients

Both client nodes use **lscr.io/linuxserver/chromium:latest**. Their web interfaces are mapped to host ports 3100/3101 and 3200/3201. Chromium was started with controlled media flags for repeatable lab testing and directed to **http://10.30.0.10:3000**.

## Verification Method and Results

Verification was performed incrementally. First, the MiroTalk container was checked from pserv. Second, the lab nodes and routing state were inspected. Third, each browser client requested the MiroTalk service through the routed topology. Finally, MarketCar opened a generated meeting room through the public HTTPS endpoint.

| Test         | Source           | Destination           | Observed result     | Meaning                            |
|--------------|------------------|-----------------------|---------------------|------------------------------------|
| Local health | pserv            | 127.0.0.1:3000        | HTTP/1.1 200 OK     | Service is running                 |
| Routed HTTP  | client1          | 10.30.0.10:3000       | HTTP/1.1 200 OK     | Path through r1 and r2 works       |
| Routed HTTP  | client2          | 10.30.0.10:3000       | HTTP/1.1 200 OK     | Path through r2 works              |
| Local HTTPS  | Browser          | https://172.16.107.5  | Media prompt worked | Secure local access was functional |
| Public HTTPS | External browser | trycloudflare.com URL | Page reachable      | Temporary external access works    |
| Integration  | MarketCar        | Generated /join URL   | Room opened         | Consumer-to-service link works     |

### Demonstration commands

```
sudo containerlab inspect -t mirotalk-final.clab.yml
docker ps --format 'table {{.Names}}\t{{.Image}}\t{{.Status}}\t{{.Ports}}'
sudo ./scripts/test-lab.sh
curl -I http://127.0.0.1:3000
```

#### Interpretation of ping results

ICMP ping was successfully used in the earlier LocalConnect proof of concept, where client1 reached 10.0.1.100 and client2 reached 10.0.2.100. The final Chromium image did not include the ping utility, so application-layer HTTP status was used as the final proof. HTTP 200 is stronger evidence for this service because it verifies routing, TCP connectivity, port 3000, and the application response together.

### Management addresses observed during the final lab run

| Node    | Management address |
|---------|--------------------|
| client1 | 172.30.30.5        |
| client2 | 172.30.30.2        |
| r1      | 172.30.30.4        |
| r2      | 172.30.30.3        |

## MarketCar Integration

MarketCar is a responsive Arabic right-to-left marketplace prototype containing vehicle listings, search, brand and price filters, seller details, and a video contact action. It represents a realistic consumer of the MiroTalk service rather than a second copy of the meeting application. Listing data and prices are demonstration data only.



Only MIROTALK\_BASE\_URL changes when the temporary tunnel URL changes.

### Integration logic

- The selected vehicle provides a stable listing identifier such as toyota-corolla-2021.
- The buyer enters a name, which is sanitized before it is used in the room identifier.
- A six-digit time-derived value reduces accidental room-name collisions during a demonstration.
- buildMeetingUrl() creates the final /join/room-id path and opens it in a new browser tab.
- noopener and noreferrer are used when opening the new tab.

```

const MIROTALK_BASE_URL = 'https://CURRENT-LINK.trycloudflare.com';

function buildMeetingUrl() {
  const roomName = `${selectedCar.id}-${buyer}-${uniqueCode}`;
  return `${MIROTALK_BASE_URL}/join/${encodeURIComponent(roomName)}`;
}
  
```

| File group           | Contents                                   |
|----------------------|--------------------------------------------|
| Interface            | index.html and style.css                   |
| Application behavior | script.js                                  |
| Local media          | Seven SVG vehicle illustrations in assets/ |
| Documentation        | README.md                                  |

### Integration design decision

The MiroTalk base address is defined once. When a Quick Tunnel restarts, only MIROTALK\_BASE\_URL must be updated; the listing buttons and room-building function remain unchanged.

# HTTPS, WebRTC, and External Demonstration Access

Camera and microphone access is not only an application setting. Browser media capture through **getUserMedia()** is restricted to secure contexts such as HTTPS or localhost, and the user must still grant permission [5]. For this reason, plain HTTP on a non-local IP is insufficient for a reliable two-device demonstration.

Two access modes were implemented. Nginx with a self-signed certificate provided local HTTPS at **https://172.16.107.5**. For access from another network or device, Cloudflare Quick Tunnel forwarded a temporary public HTTPS address to the local MiroTalk service. Quick Tunnels generate a random **trycloudflare.com** subdomain and proxy requests to a localhost web server [4].

```
cloudflared tunnel --protocol http2 --url http://127.0.0.1:3000
```

| Access mode      | Endpoint                         | Use                       | Constraint                      |
|------------------|----------------------------------|---------------------------|---------------------------------|
| Local service    | http://127.0.0.1:3000            | Server health check       | Not for remote camera use       |
| Local HTTPS      | https://172.16.107.5             | Same-network browser test | Certificate trust may be manual |
| Public HTTPS     | https://random.trycloudflare.com | Two-device demo           | URL changes after restart       |
| MarketCar local  | http://127.0.0.1:5001            | Static app on laptop      | MiroTalk still opens over HTTPS |
| MarketCar public | VS Code public dev tunnel        | Optional phone access     | Local server must be running    |

## Network restrictions encountered

ngrok and Tailscale were tested but failed under the university network's Fortinet TLS inspection. Cloudflare over HTTP/2 remained the practical temporary path. This is an environmental deployment finding, not a defect in MiroTalk or Containerlab.

### Scope of the tunnel

Cloudflare documents Quick Tunnels as testing and development tools rather than production infrastructure [4]. The project therefore uses the tunnel only for the academic demonstration. A permanent deployment should use a managed tunnel and stable domain, subject to university policy.



## Project Evolution and Engineering Decisions

The project was developed in stages so that each new layer was added only after the previous layer had been verified. This reduced troubleshooting ambiguity and provided evidence at each stage.

| Stage                | Implementation                          | Verified outcome                                 |
|----------------------|-----------------------------------------|--------------------------------------------------|
| 1. Toolchain         | WSL2, Ubuntu, Docker, Containerlab      | Containers and labs could be created             |
| 2. Proof of concept  | LocalConnect with Express and Socket.IO | Browser access, ICMP, and HTTP from two clients  |
| 3. Real-time service | MiroTalk P2P Docker image               | Video meeting application available on port 3000 |
| 4. Persistent server | Proxmox VE and Ubuntu VM pserv          | Service moved off the laptop                     |
| 5. Routed lab        | client1, r1, r2, client2, host link     | HTTP 200 from both routed clients                |
| 6. Secure access     | Nginx HTTPS and Cloudflare Quick Tunnel | Camera/microphone-ready external endpoint        |
| 7. Integration       | MarketCar room launcher                 | Vehicle listing opens a dedicated meeting room   |

### Key engineering decisions

| Decision                         | Reason                                                                                      |
|----------------------------------|---------------------------------------------------------------------------------------------|
| MiroTalk P2P instead of C2C      | Room-based interaction and broader collaboration features without SFU deployment complexity |
| Dedicated pserv VM               | Persistent service host and clear separation from the presentation client                   |
| HTTP 200 as final network proof  | Validates routing, TCP, service port, and application response                              |
| Single URL constant in MarketCar | Contains the operational effect of temporary tunnel URL changes                             |
| HTTP/2 tunnel protocol           | Avoids reliance on QUIC/UDP in a restrictive university network                             |

#### Result

The staged process transformed a three-node local proof of concept into a persistent WebRTC service with a routed four-node validation topology and a separate consuming application.

## Limitations and Future Work

The current implementation is suitable for a controlled academic demonstration. The following limitations define the boundary between the completed work and a production-ready service.

| Current limitation                       | Effect                                                                                       | Recommended next step                                                                      |
|------------------------------------------|----------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------|
| Both router instances use Nokia SR Linux | The lab has two routers but does not yet demonstrate heterogeneous network operating systems | Replace one router with FRR, VyOS, or another supported platform and repeat the same tests |
| Quick Tunnel URL is temporary            | MarketCar base URL must be updated after tunnel restart                                      | Use a managed tunnel and stable hostname when permitted                                    |
| Chromium image lacks ping                | Final ICMP output is unavailable inside those client containers                              | Install iputils or add a separate diagnostic Linux client                                  |
| Self-signed local certificate            | Other devices may not trust local HTTPS automatically                                        | Use a trusted certificate or the public HTTPS endpoint                                     |
| No dedicated TURN validation             | Peer-to-peer media may fail between highly restrictive NATs                                  | Deploy coturn and test relay candidates                                                    |
| Demo-scale measurement                   | The work verifies functionality, not capacity                                                | Measure latency, packet loss, setup time, and concurrent sessions                          |

### Security and operational improvements

- Protect hosted rooms with authentication or signed room credentials before real public use.
- Restrict CORS and proxy trust settings to the required origins rather than broad development values.
- Run cloudflared as a managed service only after a permanent tunnel is approved.
- Add logging and lightweight monitoring for service status, tunnel status, and failed room joins.
- Document a recovery sequence for restarting MiroTalk, rebuilding the lab, and validating both clients.

#### Highest-priority technical extension

Use a different network operating system for one router, then capture the route tables and repeat client1/client2 HTTP checks. This directly strengthens the network-programming evidence without changing the application layer.

The project achieved its main objective: a real-time network service was deployed, routed, exposed securely for demonstration, and consumed by another application. MiroTalk P2P runs on the pserv Ubuntu VM under Proxmox VE and responds on port 3000. The Containerlab environment separates the clients into distinct networks and routes traffic through r1 and r2 to the host-side service network. Both final clients received HTTP 200 responses from MiroTalk, confirming application-level reachability across the configured paths.

MarketCar demonstrates service integration rather than simple co-location. A user selects a vehicle, provides a name, and receives a listing-specific MiroTalk room URL generated by the marketplace. The meeting opens through an HTTPS endpoint, satisfying the secure-context requirement for camera and microphone permission requests. The final system therefore connects infrastructure, routing, application deployment, browser security, and software integration in one coherent network-programming project.

| Outcome                                  | Status           |
|------------------------------------------|------------------|
| Self-hosted WebRTC service on Proxmox    | Completed        |
| Docker-based service operation           | Completed        |
| Two-client routed Containerlab topology  | Completed        |
| HTTP reachability from both clients      | Completed        |
| Temporary HTTPS access for browser media | Completed        |
| MarketCar consumer integration           | Completed        |
| Two different router operating systems   | Future extension |

## Final statement

This is not only a deployed meeting website. It is a reusable communication service running on dedicated virtualized infrastructure, reachable through a defined routed topology, and invoked by a separate marketplace application.

## References

- [1] MiroTalk, MiroTalk P2P - Self-Hosted Open-Source WebRTC Video Conferencing. Available: [github.com/miroslavpejic85/mirotalk](https://github.com/miroslavpejic85/mirotalk). Accessed Aug. 2, 2026.
- [2] Containerlab, Container-Based Networking Labs. Available: [containerlab.dev](https://containerlab.dev). Accessed Aug. 2, 2026.
- [3] Proxmox Server Solutions, Proxmox Virtual Environment Overview. Available: [proxmox.com](https://proxmox.com). Accessed Aug. 2, 2026.
- [4] Cloudflare, Quick Tunnels. Available: [developers.cloudflare.com](https://developers.cloudflare.com). Accessed Aug. 2, 2026.
- [5] MDN Web Docs, MediaDevices: getUserMedia() Method. Available: [developer.mozilla.org](https://developer.mozilla.org). Accessed Aug. 2, 2026.